Chapitre 2: Fonctions et mécanismes cryptographiques

Ouissem Ben Fredj
Ouissem.benfredj@gmail.com

Plan

1. Éléments théoriques de la cryptographie
2. Algorithmes de chiffrement symétriques & asymétriques & variantes.
3. Fonctions de hachage, Signature numérique, scellement
4. Performances des fonctions cryptographiques vs contraintes des objets connectés
5. Hiérarchie des clés & échange des clés
6. Certificats et PKIs

définitions

La **cryptographie** est une discipline consistant à manipuler des données de telle façon que les **services** suivants puissent être fournis :

- **Intégrité**
  - Objectif : s'assurer que les données n'ont pas été modifiées sans autorisation. Remarque : dans les faits, la cryptographie ne s'attache pas vraiment à empêcher une modification de données, mais plutôt à fournir un moyen sûr de détecter une modification malveillante.
- **Confidentialité**
  - Objectif : ne permettre l'accès aux données qu'aux seules personnes autorisées.
- **Preuve** (**authentification** et **non-répudiation**)
  - Objectif : fournir un moyen de preuve garantissant la véritable identité des entités ainsi que l'imputation de leurs actions.

Vocabulaire de base

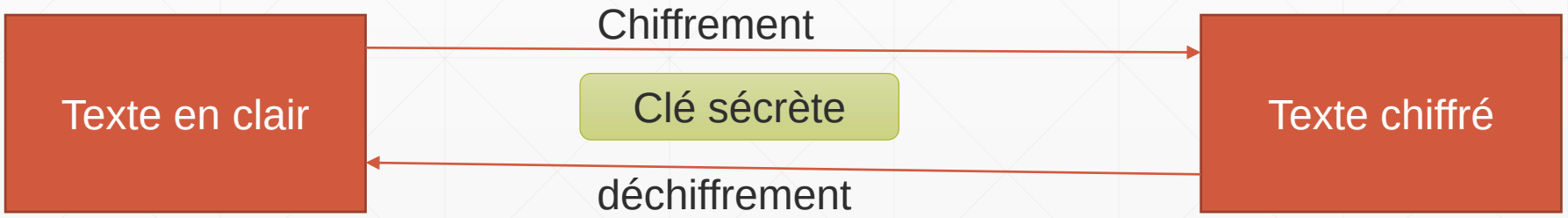
- **Texte en clair** : l'objet (texte, document, image, . . .) à chiffrer.
- **Texte chiffré** : résultat du chiffrement.
- **Chiffrement** : processus a travers lequel un texte en clair est converti en un texte chiffré.
- **Algorithme de chiffrement** : étapes de traitement des données permettant de transformer un texte en clair en un texte chiffré.
- **Clé secrète** : paramètre utilise par l'algorithme de chiffrement.
- **Déchiffrement** : processus permettant d'obtenir le texte en clair à partir du texte chiffré.
- **Algorithme de déchiffrement** : étapes de traitement des données permettant de transformer un texte chiffré en un texte en clair.

Vocabulaire de base

- **Cryptographie** : schémas et mécanismes disponibles pour le chiffrement et le déchiffrement.
- **Système cryptographique (cryptosystème)** : se compose de trois algorithmes: un pour la génération de clé, un pour le cryptage et un pour le décryptage..
- **Cryptogramme** : texte chiffré a l'aide d'un cryptosystème.
- **Chiffrement par bloc** : transformation à la fois d'un bloc de données en entrée en un bloc chiffré de la même taille.
- **Chiffrement par flot** : chiffrement des données à la volée et n'a pas besoin de les découper.

Vocabulaire de base

- **Cryptanalyse** : signifie "casser un code". Elle se base sur la connaissance de l'algorithme de chiffrement et une certaine connaissance de la structure probable du texte en clair. Ceci afin de reconstruire partiellement ou totalement le texte en clair a partir du texte chiffré ou déduire la clé de déchiffrement.
- **Cryptologie** : science qui regroupe la cryptographie et la cryptanalyse.
- **Espace des clés** : nombre total de toutes les clés possible pouvant être utilisées dans un système cryptographique.



Principe de Kerckhoff

« L'opposant connaît le système cryptographique

&

Toute la sécurité d'un système cryptographique doit reposer sur la clé, et pas sur le système lui-même »

Auguste Kerckhoff, "La cryptographie militaire", février 1883

Chiffrement symétrique

- Chiffrement à clé sécrète ou privée.
- La clé de cryptage et la clé de décryptage sont les mêmes et donc doivent être gardées sécrètes.
- Transformations similaires pour codage et décodage (protocoles symétriques).
- Dans cette méthode de chiffrement, on distingue deux types d'algorithmes :
  - l'algorithme **par bloc** qui prend une longueur spécifiée de données comme entrée, et produit une longueur différente de données chiffrées (exemple : DES, AES…)
  - l'algorithme en **flux continu** qui chiffre les données un bit à la fois (exemple : IDEA, CAST, RC4, SKIPjack…).

Chiffrement par bloc symétrique

1. coder l'information source en binaire. On obtient ainsi une chaine de caractères composée de 0 et de 1.
2. découper cette chaîne en blocs de longueur donnée (par exemple 64 bits ou 128 bits ou 256 bits).
3. chiffrer un bloc en faisant un OU exclusif (XOR) bit a bit avec une clé secrète, k, qui est une suite de 0 et de 1 de même longueur.
4. déplacer et permuter certains bits du bloc.
5. recommencer un certain nombre de fois l‘étape précédente, on appelle cela une ronde.
6. passer au bloc suivant et retourner a l‘étape 3 jusqu' a ce que tous les blocs soient chiffrés.

Algorithmes de chiffrement symétrique

- DES Data Encryption Standard 1976
- TDES Triple Data Encryption Standard 1990
- IDEA International Data Encryption Algorithm 1991
- Blowfish Coup de poisson 1993
- RC5 Rivest Cipher 1994

Chiffrement symétriques: exemples

	DES	AES
Taille de clé	56 bits	128, 192 ou 256 bits
Type de chiffre	Chiffre à bloc symétrique	Chiffre à bloc symétrique
Taille de bloc	64 bits	128, 192 ou 256 bits
Résistance à la cryptanalyse	Vulnérable à la cryptanalyse linéaire et différentielle; tables de substitution faibles	Résistant contre des attaques différentielles, linéaires et par interpolation.
Sécurité	Prouvé comme inadéquat	Considéré sécurisé
Clés possibles	2^{56}	2^{128},2^{192} ou 2^{256}
Clés possibles composées de 95 caractères ASCII affichables	95^{7}	95^{16}, 95^{24} ou 95^{32}
Recherche sur toutes les clés possibles à 50 milliard de clés par seconde	Pour une clé de 56 bits: 400 jours	Pour une clé de 128 bits: 5 x 1021 years

Inconvénients d'un cryptosystème à clé symétrique

- Le chiffrement symétrique repose sur l'échange d'un secret (les clés).
- Lors d'échange entre plusieurs intervenants : une clé est partagée que par 2 interlocuteurs, donc pour N interlocuteurs il faut N*(N-1)/2 clés.
- Pour DES, La taille de la clé est petit (56 bits), une attaque de force brute est possible.

Chiffrement asymétrique

- Le principe de chiffrement asymétrique (appelé aussi chiffrement à clés publiques) est apparu en 1976.
- Les clés existent par paires (le terme de bi-clés est généralement employé) :
  - Une **clé publique** pour le chiffrement.
  - Une **clé secrète** pour le déchiffrement.
- Les utilisateurs s‘échangent cette clé publique au travers d'un canal non sécurisé.
- Algorithmes de chiffrement asymétrique
  - **RSA** : est algorithme décrit en 1977 par Ronald Rivest, Adi Shamir et Leonard Adleman (le plus utilise).
  - Cryptosysteme de **ElGamal** : est un algorithme crée par Taher Elgamal en 1984.
  - Cryptosysteme de **Merkle-Hellman**: est un algorithme défini par Ralph Merkle et Martin Hellman en 1978.

Chiffrement par bloc asymétrique

1. coder l'information source en binaire. On obtient ainsi une chaîne de caractères composée de 0 et de 1.
2. découper cette chaîne en blocs de longueur donnée (par exemple 1024 bits a 2048 bits pour RSA et El Gamal, 256 bits pour les codes elliptiques (des échanges de clés sur un canal non-sécurisé ou un chiffrement asymétrique)).
3. chiffrer un bloc en utilisant la fonction de chiffrement (exponentiation modulaire pour RSA).
4. passer au bloc suivant et retourner a l‘étape 3 jusqu'a ce que tous les blocs soient chiffres.

RSA

- Rivest Shamir Adleman ou RSA est un algorithme asymétrique de cryptographie à clé publique,
  - très utilisé dans le commerce électronique,
  - et plus généralement pour échanger des données confidentielles sur Internet.
- Les clefs RSA sont habituellement de longueur comprise entre 1024 et 2048 bits.
- Principe:
  - Il est facile de fabriquer deux grands **nombres premiers** p et q (>100)
  - étant donné un nombre entier n = pq, il est très difficile de retrouver les facteurs p et q

RSA: validité d’un chiffrage

- Pour les clés secrètes, la référence est la recherche exhaustive (dépend de la longueur de la clé, 2^{n-1} opérations) :
- Complexité en temps : $2^{128} = 10^{40}$ opérations.
- Moyens : Un million de stations travaillant à un milliard d’opérations par seconde (10^{15}).
- Temps requis : Avec 10^{15} opérations par seconde, il faut 10^{25} secondes, c’est-à-dire de l’ordre d’un milliard de milliards d’années.

RSA

- Création des clés :
  - Soient **p et q** deux grands nombres premiers, n=pq, Soit φ(n) = (p - 1)(q - 1)
  - calculer **e:** e est premier avec φ(n) c-à-d pgcd(e, φ(n))=1 c-à-d e et φ(n) n'ont que 1 et -1 comme diviseurs en commun
  - calculer **d** tel que e * d = 1 mod φ(n) ou e * d mod φ(n) = 1 (en utilisant l'algorithme d'Euclide étendu)
  - **La clé publique**: (n, e)
  - **La clé secrète**: (n, d)
- **Chiffrement** :
  - Le chiffrement d'un message M en un message codé C se fait suivant la transformation suivante :
  - **$C = M^e \bmod n$**
- **Déchiffrement** :
  - il s'agit de calculer la fonction réciproque
  - **$M = C^d \bmod n$**

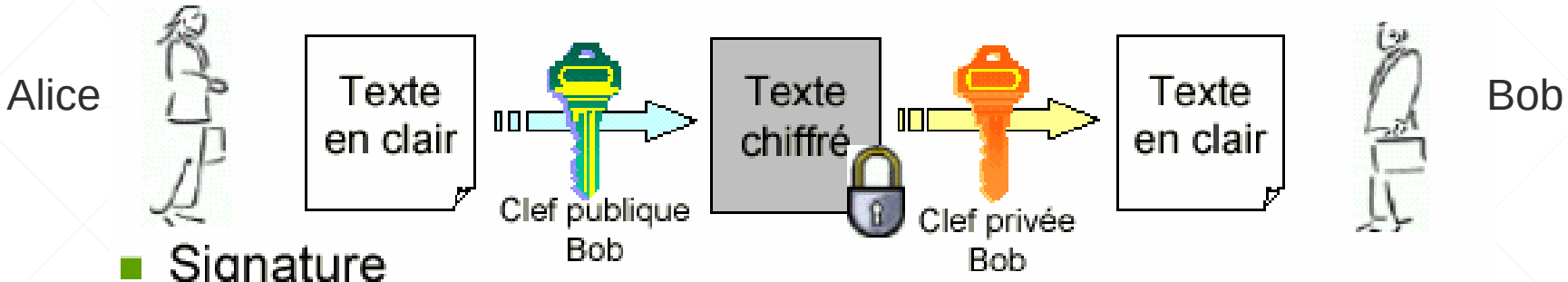
RSA: les blocs

- On transforme d'abord les lettres du message en nombres et ceci en remplaçant chaque lettre par son rang dans l'alphabet.
- On découpe ensuite le message chiffré en blocs de même longueur mais **plus petit que n**.
- Quelque nombres premiers:

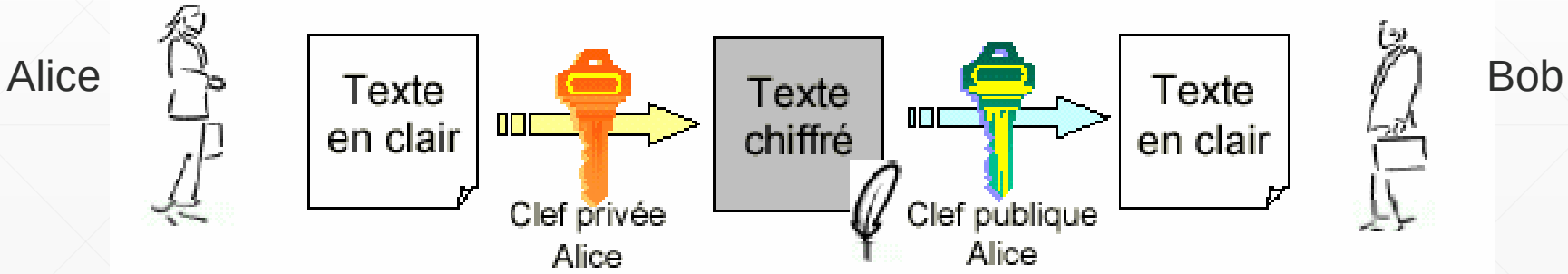
2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	59	61	67	71
73	79	83	89	97	101	103	107	109	113	127	131	137	139	149	151	157	163	167	173
179	181	191	193	197	199	211	223	227	229	233	239	241	251	257	263	269	271	277	281
283	293	307	311	313	317	331	337	347	349	353	359	367	373	379	383	389	397	401	409

Principe du chiffrement asymétrique

- Chiffrement
  - Clef publique utilisée pour le chiffrement, seul le détenteur de la clef privée peut déchiffrer

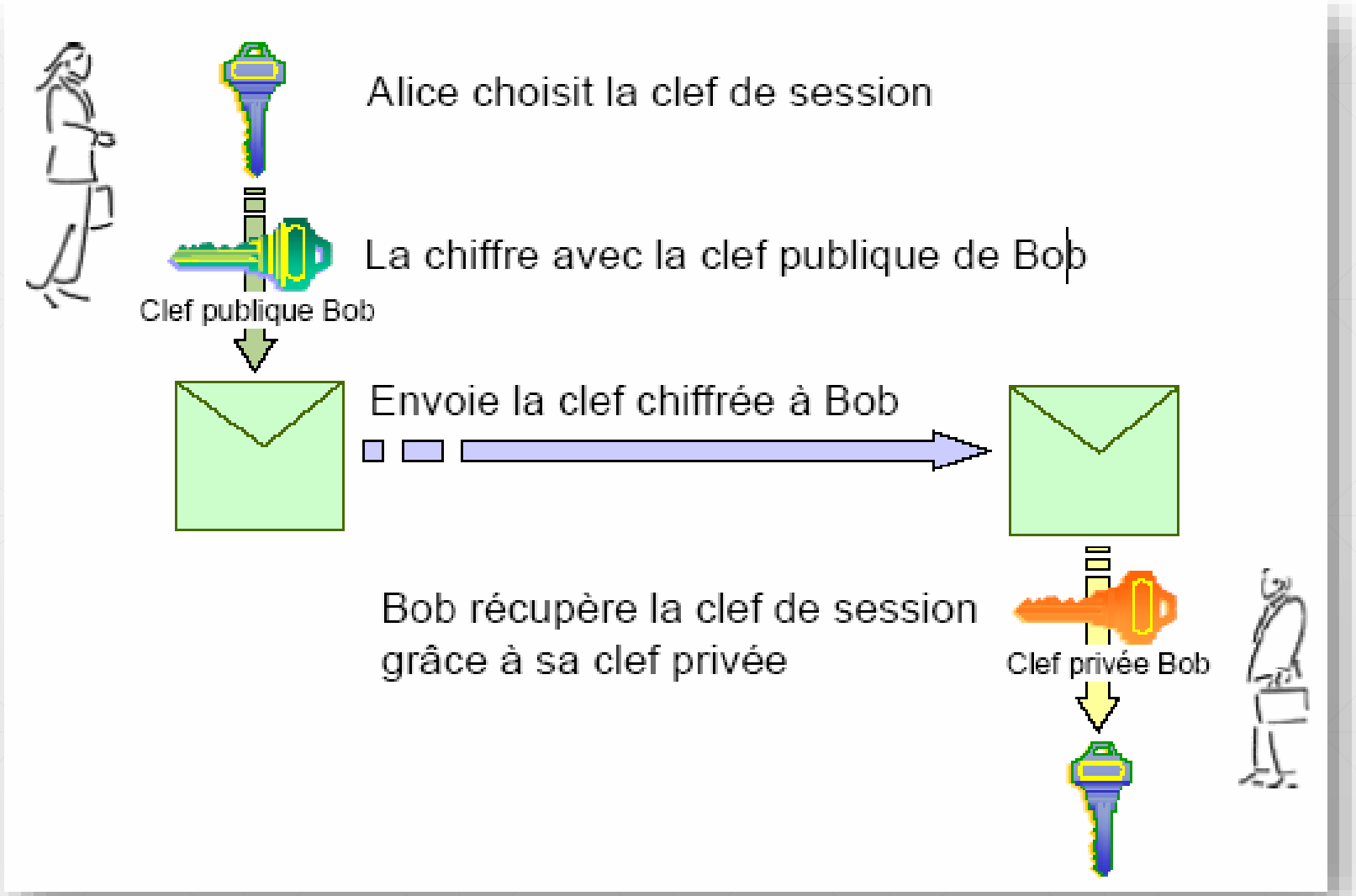


- Signature
  - Clef privée utilisée pour le chiffrement, seul son détenteur peut chiffrer, mais tout le monde peut déchiffrer (et donc en fait vérifier la "signature")



Transport de clés de session avec clé publique

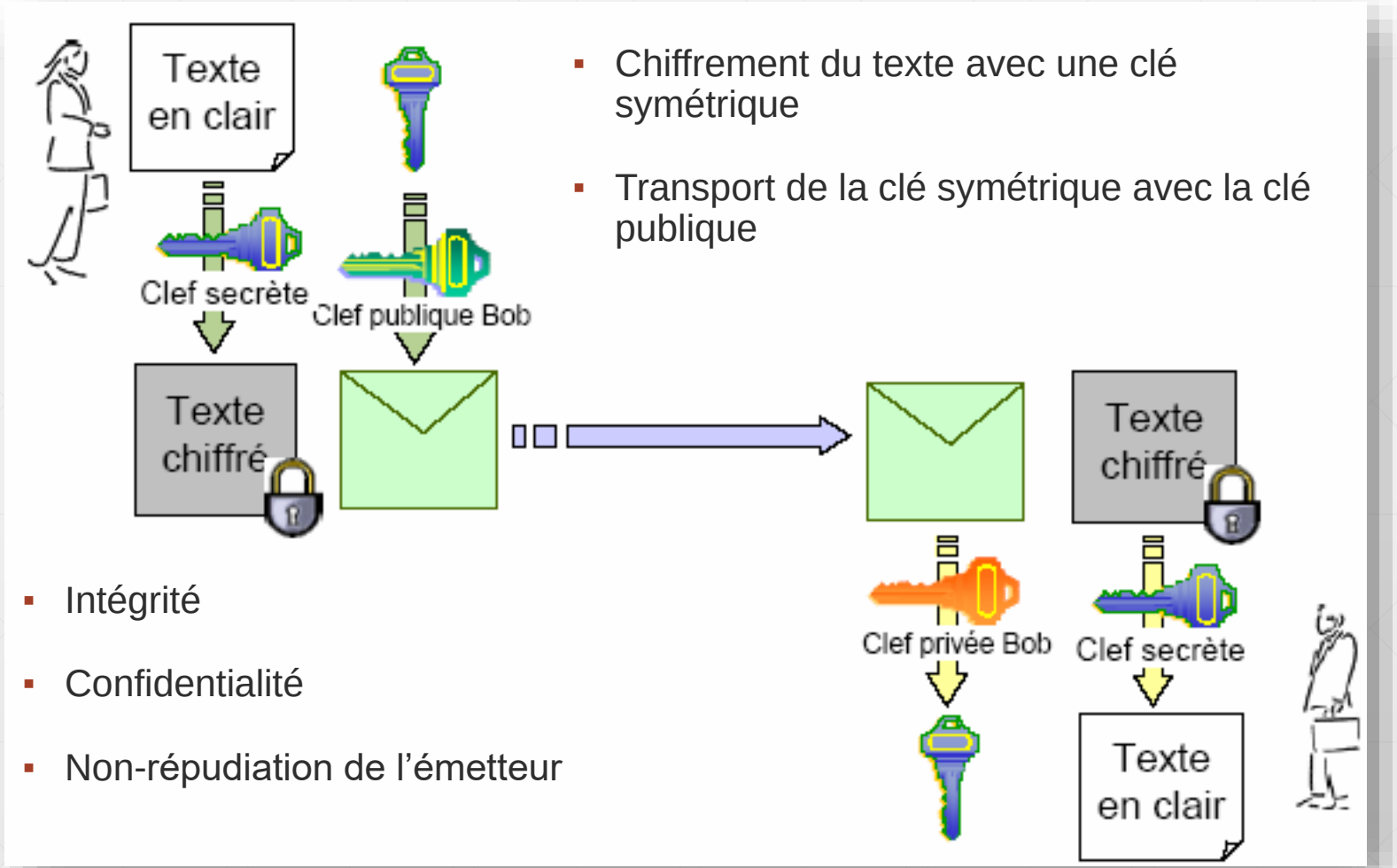
Alice



Bob

Exemple d'utilisation: clé symétrique et clés privés

Alice



- Chiffrement du texte avec une clé symétrique
- Transport de la clé symétrique avec la clé publique

- Intégrité
- Confidentialité
- Non-répudiation de l'émetteur

Bob

Echange avec clé asym: Avantages et Inconvénients

- Le problème consistant a se communiquer la clé de déchiffrement n'existe plus, dans la mesure ou les clés publiques peuvent être envoyées librement.
- Le chiffrement par clés publiques permet donc à des personnes d‘échanger des messages chiffrés sans pour autant posséder de secret en commun.
- En contrepartie, tout le challenge consiste a s'assurer que la clé publique que l'on récupère est bien celle de la personne à qui l'on souhaite faire parvenir l'information chiffrée !

Diffie-Hellman: protocole d'Échange de clés

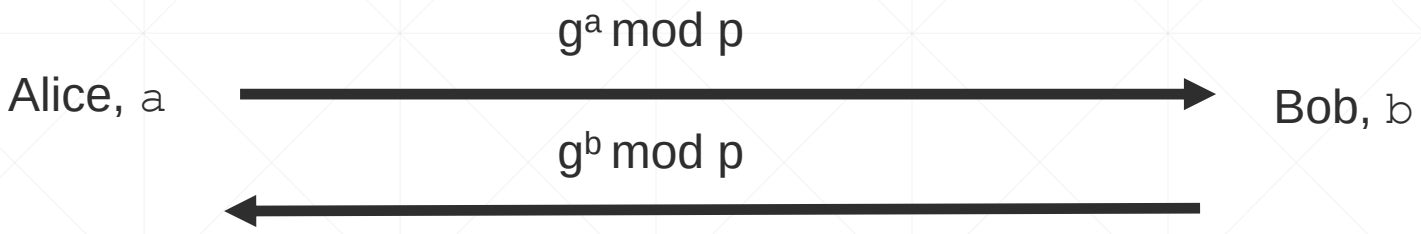
- Inventé par Williamson (GCHQ) et, indépendamment, par D et H (Stanford)
- Un algorithme « d'échange de clés »
  - Utilisé pour établir **une clé symétrique partagée**
  - ***Pas*** pour chiffrer ou signer
- Basé sur le problème de **log discret**
  - **Donné:** g , p et g^k mod p
  - **Trouver :** exposant k

Diffie-Hellman

- Soit p premier , soit g générateur
  - Pour tout $x \in \{1,2,\ldots,p-1\}$ il y a **n** t.q. $\mathbf{x} = g^{n} \bmod p$
- Alice sélectionne sa valeur privée **a**
- Bob sélectionne sa valeur privée **b**
- Alice envoie $\mathbf{g^{a}}$ **mod** p à Bob
- Bob envoie $\mathbf{g^{b}}$ **mod p** à Alice
- Les deux calculent le secret partagé, $\mathbf{g^{ab}}$ **mod p**
- Le secret partagé peut être utilisé comme clé symétrique

Diffie-Hellman

- **Public:** g et p
- **Privé :** l'exposant **a** d'Alice , l'exposant **b** de Bob



- Alice calcule $(g^b)^a = g^{ba} = g^{ab}$ mod p
- Bob calcule $(g^a)^b = g^{ab}$ mod p
- Ils peuvent utiliser $K = g^{ab}$ mod p comme clé symétrique

Diffie-Hellman

- Supposons que Bob et Alice utilisent Diffie - Hellman pour déterminer la clé symétrique $K = g^{ab} \bmod p$
- Trudy peut voir $g^a \bmod p$ et $g^b \bmod p$
  - Mais… $g^a g^b \bmod p = g^{a+b}$ mode $p \neq g^{ab} \bmod p$
- Si Trudy peut trouver a ou b , elle obtient K
- Si Trudy peut résoudre le problème de **log discret**, elle peut trouver a ou b

Diffie-Hellman

- Soumis à une attaque de Man-In-the-Middle (MiM)

- Trudy partage le secret g^{at} mod p avec Alice
- Trudy partage le secret g^{bt} mod p avec Bob
- Alice et Bob ne savent pas que Trudy est MiM

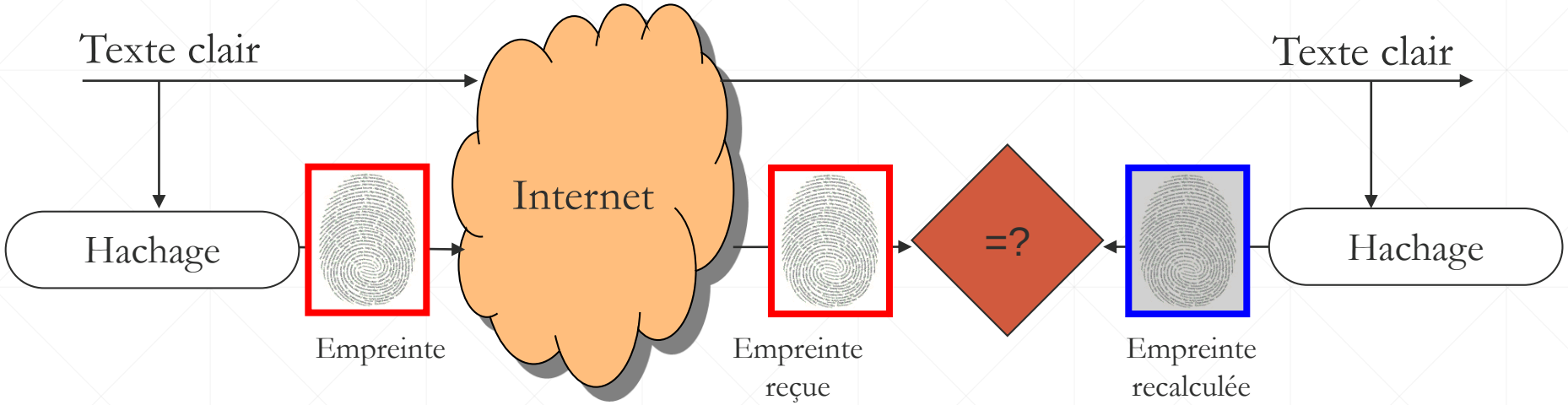
Diffie-Hellman

- Comment prévenir l'attaque MiM ?
  - Crypter l'échange DH avec une clé symétrique
  - Crypter l'échange DH avec une clé publique
  - Signer les valeurs DH avec une clé privée
  - Autre?
- Vous **DEVEZ** avoir attention de l' attaque MiM sur Diffie-Hellman

Fonction de hachage

- **Entrée**: message M avec contenu et taille arbitraire.
- **Sortie**: message de taille fixe h=H(M).
- La fonction de hachage permet d'extraire une empreinte qui caractérise les données.
  - Une empreinte a toujours une taille fixe indépendamment de la taille des données.
- **Irréversible**:
  - Etant donnée h, il est difficile de trouver x tel que: h = H(x)
  - Complexité de l'ordre de 2^n, n est le nombre de bits du *digest*.
- Calcul facile et rapide (plus rapide que le cryptage symétrique).
- Exemples:
  - MD5, SHA, …
  - Taille du *digest*: 128-160-… bits

Fonctions de Hachage: cas d'utilisation



1)

Le texte reçu est intègre

2)

Le texte reçu est altéré

Propriétés d’une fonction de hachage

- **Irréversible**
  - Soit « y » le résultat de hachage, il est pratiquement infaisable de trouver « x » tel que h(x)=y.
- **Résistance forte à la collision**:
  - Soit « x » et « y=h(x) », il est pratiquement infaisable de trouver «x’≠x » tel que h(x’)=h(x).
  - Il est pratiquement infaisable de trouver deux valeurs distinctes « x’ » et « x » tel que h(x’)=h(x).

Propriété de fonction de Hachage

Tableau. Exemples d'empreintes générées par l'algorithme MD5

cette phrase se termine par a	30aa5e7e92a8c71cc10fe9eeebe01a3b
cette phrase se termine par e	d7e1076cbe1d660e14f384683d82cc06
cette phrase se termine par 3	e93dde961b27878ae858c8e2f696aa7f
cette phrase se termine par *	828b883f6a0ec783181a6475c0aeefd2

Cet exemple montre qu'un petit changement au niveau du texte donne des empreintes totalement différente. Et donc c'est la similarité entre deux empreinte ne reflète pas une similarité entre ses correspondant textes.

Fonctions de Hachage: Exemples

- **MD5** : *Message Digest 5*
  - Développé en 1991
  - Génère une empreinte de taille 128 bits en traitant les données d'entrée par blocs de 512 bits.
- **SHA-1** : *Secure Hash algorithm*
  - Génère une empreinte de taille 160 bits.
  - Plus fort que MD5.
- **SHA-256**
  - Génère une empreinte de taille 256 bits
  - Utilisé par Bitcoin

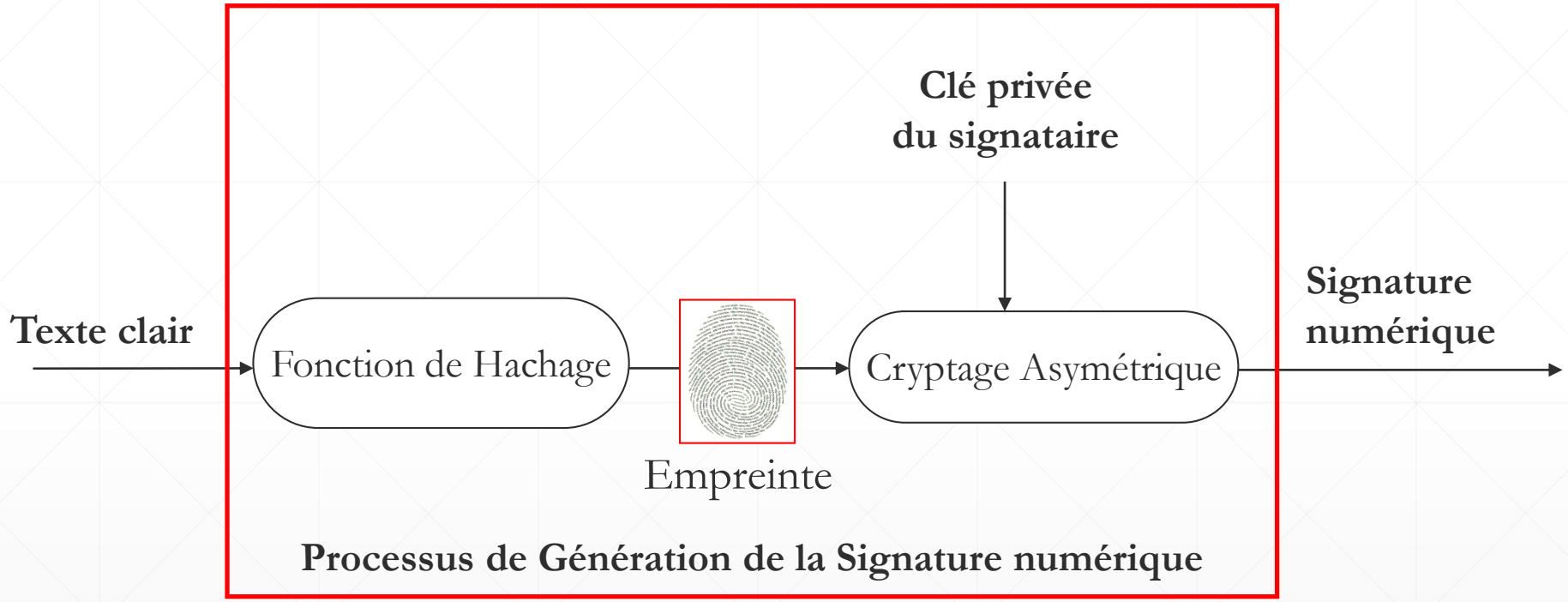
Signature numérique symétrique (scellement)

- L’émetteur calcule le digest (hash) sur la concaténation de la clé secrète et du message, et envoi le digest et le message
- Le destinataire procède de même :
  - Si digest trouvé est identique à celui reçu, alors :
    - le message n’a pas été altéré (intégrité),
    - il a bien été émis par l’autre détenteur de la clé partagé (non répudiation de l’émetteur).
- Mécanisme qui fournit les services suivants :
  - Authentification de l‘origine des données
  - Intégrité
  - ne fournit pas la non-répudiation coté récepteur

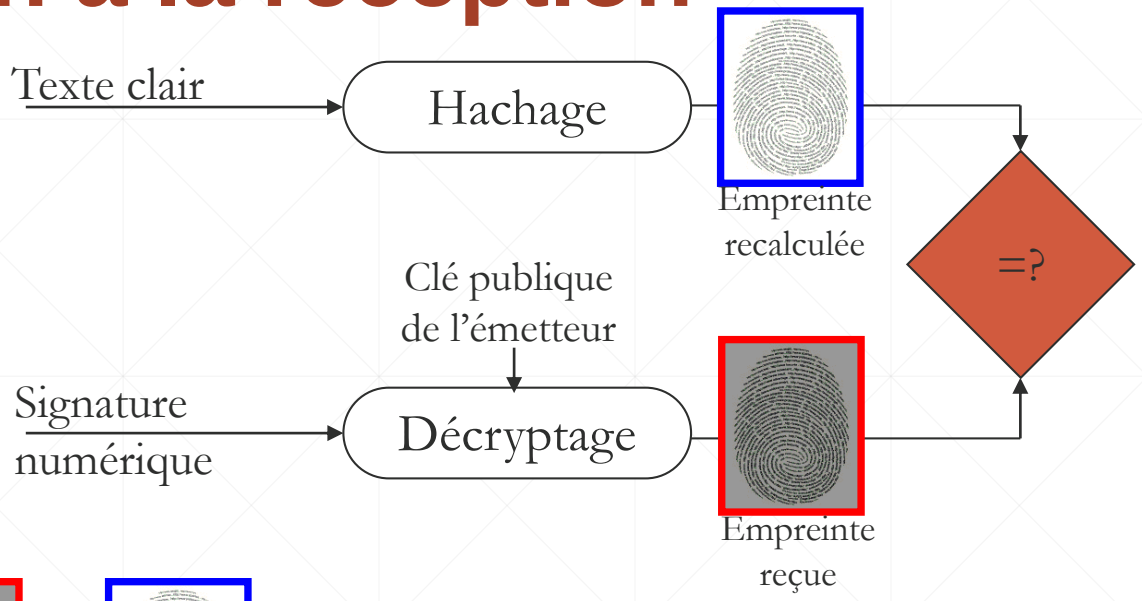
Signature numérique asymétrique

- Principe de fonctionnement
  - Le ***Hash*** (résultat de la fonction de hachage) d'un message est crypté avec la **clé privée de l'émetteur**.
  - La **clé publique** est utilisée pour la **vérification** de la signature
- Soit:
  - M: message à signer, H: fonction de hachage
  - Kpr, Kpu: paire de clés privée/publique de l'émetteur.
  - E / D: fonction de cryptage / Décryptage en utilisant Kpu / Kpr.
- En recevant (M, $E_{Kpr}(H(M))$), le récepteur vérifie si: $H(M)=D_{Kpu}(E_{Kpr}(H(M)))$

Signature numérique asymétrique: Génération



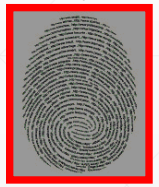
Signature numérique asymétrique : Vérification à la réception



1) =

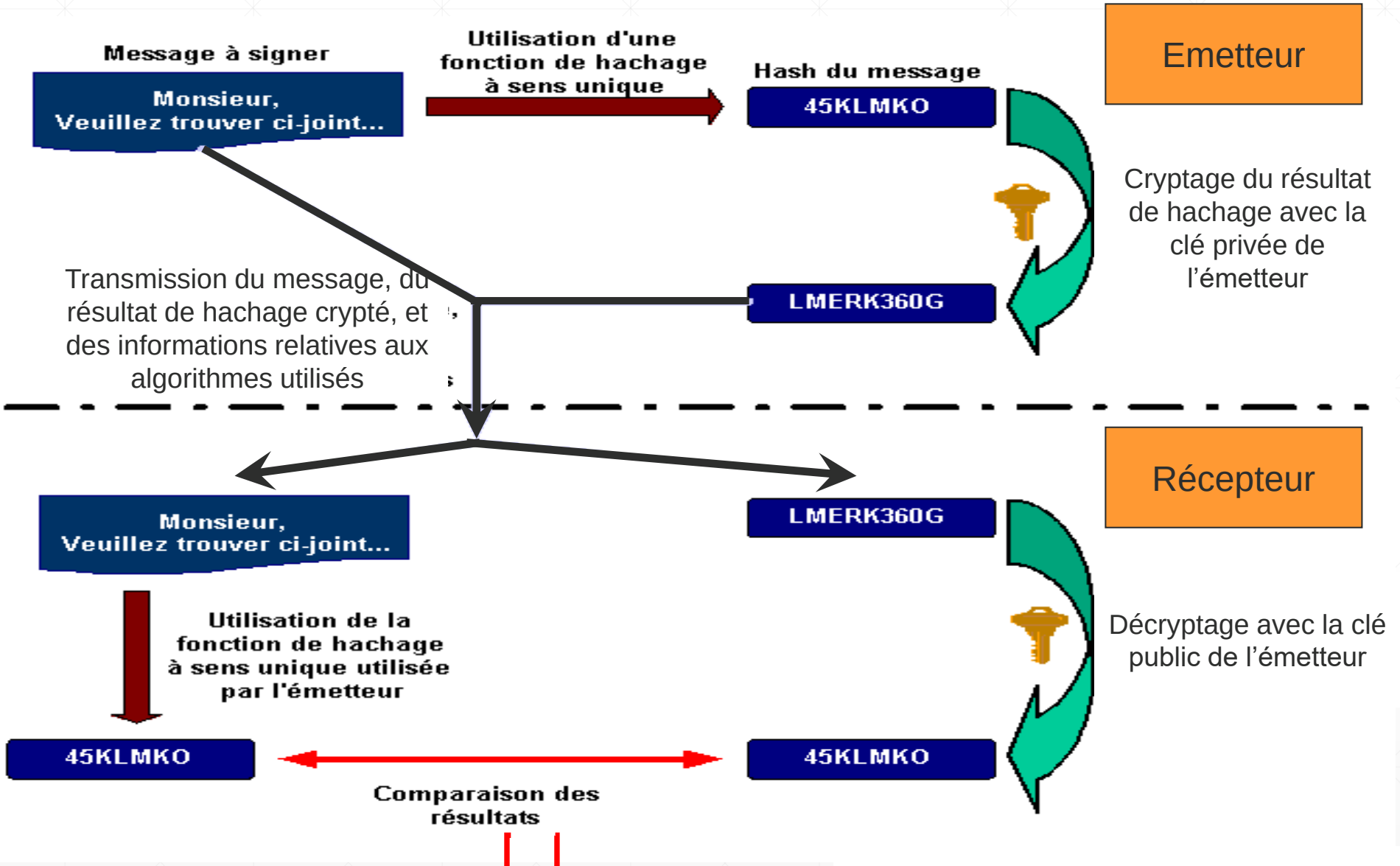
Empreinte reçue — Empreinte recalculée

La signature reçue est correcte

2)  ≠

Empreinte reçue — Empreinte recalculée

La signature reçue est incorrecte



Signature incorrecte

Résultats différent

Résultats égaux

1. **Emetteur du message authentifié**
2. **message intègre**
3. **l'émetteur ne peut pas répudier l'envoi**

Signature numérique asymétrique

- Mécanisme qui fournit les services suivants :
  - Authentification de l‘origine des données
  - Intégrité
  - Non-répudiation de la source

Distribution des clés publiques

- **Idée de départ** : consiste à utiliser un simple annuaire des clés publiques.
- **Problèmes** : Distribuer les clés d'une façon authentifié et intègre et limiter le nombre de clés à stocker.
- **Solution** : Utiliser les certificats et la hiérarchie de certification

Autorités de certification (A.C.)

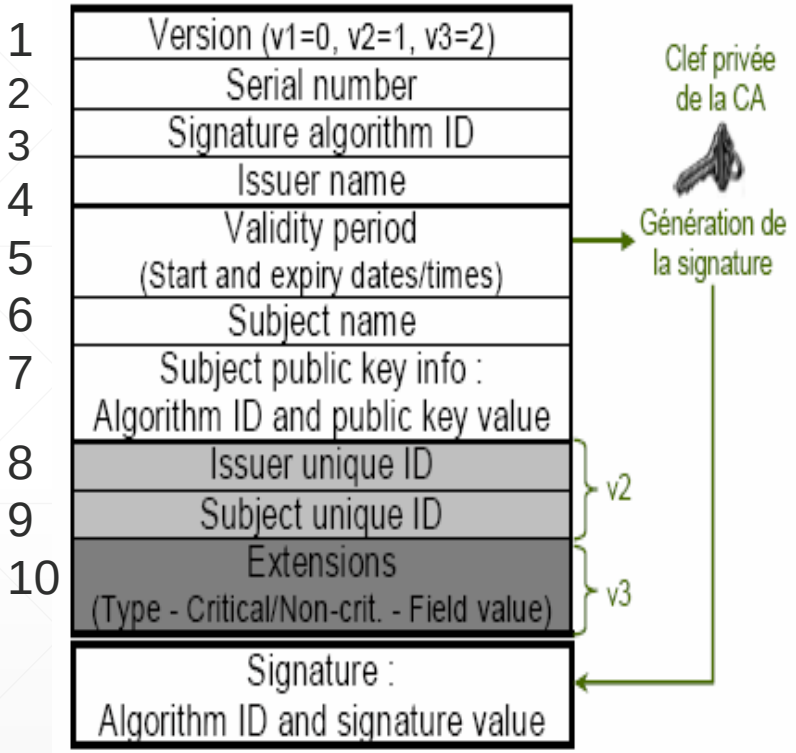
- Ce sont des infrastructures à clés publique(PKI: Public Key Infrastructure)
- Rôle : gestion des clés publiques à grande échelle (moyennant des certificats)
- Enregistrement et émission
- Stockage et distribution
- Révocation et vérification de statut
- Sauvegarde et récupération
- Exemple : A.N.C.E, Symantec (Verisign)

Les Certificats

- Un certificat est l'élément de transport d'une clé publique
- Un certificat est émise par une A.C
- L'authenticité du certificat est vérifiable d'une façon autonome
  - toute modification du certificat sera détectée
- Le certificat permet **l'authentification** du possesseur
  - lie la clé publique au possesseur

Structure du certificat X.509v3

1. Indique à quelle version X.509 correspond ce certificat
2. Numéro de série (propre à chaque autorité de certification)
3. Identifiant du type de signature utilisée
4. Nom distingué de l'autorité de certification qui a émis ce certificat
5. Période de validité
6. Nom distinguée du détenteur de la clé publique
7. Informations sur la clé publique de ce certificat
8. Extensions optionnelles introduites avec la version 2 de X.509
9. Extensions : extensions génériques optionnelles, introduites avec la version 3 de X.509
10. Signatures numériques de l'autorité de certification su l'ensemble des champs précédents



La signature du certificat

- La génération de la signature du certificat s‘effectue en deux étapes :
  - Un "message digest" est générée par hachage sur les données du certificat en utilisant l'algorithme SHA-1 ou MD5, le plus utilisé étant SHA-1.
  - Le haché (ou l’empreinte) est ensuite cryptée avec la clé privée de la CA qui génère le certificat.
- Vérification de la signature :
  - le destinataire, qui connaît la clé publique de signature de CA, l’utilise pour retrouver la valeur du haché du certificat. Cette opération permet de s'assurer de l'identité de l‘émetteur (CA).
  - ensuite, le destinataire calcule le haché du certificat à partir de message reçu. Si les deux empreintes sont identiques, cela signifie que les données signées sont intègres et authentifiées.

Étapes pour envoyer un message

- L’émetteur signe le message M avec sa clé privée (résultat: Ms)
- L’émetteur crypte le message M avec la clé publique du récepteur (résultat: C)
- L’émetteur envoie Ms et C au récepteur
- Le récepteur vérifie la signature de M (Ms) avec la clé publique de l’émetteur (son certificat)
- le récepteur déchiffre le message envoyé par sa clé privée.